```
        self.linear1 = nn.Linear(self.node_size, self.node_size)
        self.norm1 = nn.LayerNorm([self.N,self.node_size],
    elementwise_affine=False)
        self.linear2 = nn.Linear(self.node_size, self.out_dim)
```

- ***1*** Defines the number of channels for each convolutional layer
- ***2*** self.H and self.W are the height and width of the input image, respectively.
- ***3*** The dimension of the nodes after passing through the relational module
- ***4*** The number of objects or nodes, which is just the number of pixels after passing through the convolutions
- ***5*** The dimensionality of each node vector is the number of channels in the last convolution plus 2 spatial dimensions.
- ***6*** Layer normalization improves learning stability.

The basic setup of our model is an initial block of four convolutional layers that we use to preprocess the raw pixel data into higher-level features. Our ideal relational model would be completely invariant to rotations and smooth deformations, and by including these convolutional layers that are only translation-invariant, our whole model is now less robust to rotations and deformations. However, the CNN layers are more computationally efficient than relational modules, so doing some preprocessing with CNNs usually works out well in practice.

After the CNN layers, we have three linear projection layers that project a set of nodes into a higher-dimensional feature space. We also have some LayerNorm layers (discussed in more detail shortly), and a couple of linear layers at the end. Overall, it’s not a complicated architecture, but the details are in the forward pass of the model.

Listing 10.3. The forward pass (continued from listing 10.2)

```
def forward(self,x):
        N, Cin, H, W = x.shape
        x = self.conv1(x)
        x = torch.relu(x)
        x = self.conv2(x)
        x = x.squeeze()
        x = torch.relu(x)
        x = self.conv3(x)
```